

UAO Finder: Sistema de Navegación Inteligente para Campus Universitario mediante Visión por Computadora y Algoritmos de Ruta Más Corta

Plantilla de Reporte Estilo IEEE

Resumen

Este trabajo presenta UAO Finder, un sistema de navegación inteligente para el campus de la Universidad Autónoma de Occidente (UAO) en Cali, Colombia. El sistema combina técnicas de visión por computadora basadas en Faster R-CNN para la detección de puntos de referencia con el algoritmo de Dijkstra para la búsqueda de rutas más cortas en un grafo ponderado que modela la red peatonal del campus. Se recopiló un dataset de más de 1,236 imágenes distribuidas en 6 clases de puntos de referencia. La aplicación móvil fue desarrollada con React Native y Expo, proporcionando una experiencia multiplataforma (iOS/Android) con mapa interactivo SVG e instrucciones de navegación paso a paso.

Palabras clave: Visión por computadora, Faster R-CNN, Dijkstra, navegación indoor, React Native, campus universitario.

1. Introducción

La orientación en campus universitarios representa un desafío significativo para estudiantes de primer ingreso. Los campus de gran extensión, como el de la UAO, cuentan con múltiples edificios, zonas de servicios y áreas comunes que pueden resultar confusos para quienes no están familiarizados con la distribución espacial del lugar.

Este proyecto propone una solución tecnológica que combina:

1. **Detección visual de puntos de referencia** mediante modelos de deep learning
2. **Cálculo de rutas óptimas** mediante algoritmos de grafos
3. **Interfaz móvil intuitiva** con mapa interactivo y direcciones paso a paso

El objetivo es proporcionar una herramienta accesible que permita a los estudiantes nuevos navegar de manera autónoma y eficiente dentro del campus.

2. Marco Teórico

2.1 Faster R-CNN

Faster R-CNN (Region-based Convolutional Neural Network) es una arquitectura de detección de objetos propuesta por Ren et al. (2015). Se compone de:

- **Backbone CNN:** Red convolucional (ResNet-50 o MobileNetV2) para extracción de características

- **Region Proposal Network (RPN):** Genera propuestas de regiones candidatas donde podrían existir objetos
- **ROI Pooling:** Alinea las regiones propuestas a un tamaño fijo para clasificación
- **Cabezas de Clasificación y Regresión:** Predicen la clase del objeto y refinan las coordenadas del bounding box

Para este proyecto, se emplean dos variantes de backbone:

- **ResNet-50:** Mayor precisión, adecuado para entrenamiento y evaluación
- **MobileNetV2:** Menor tamaño y latencia, optimizado para inferencia en dispositivos móviles mediante TensorFlow Lite

2.2 Algoritmo de Dijkstra

El algoritmo de Dijkstra (1959) encuentra la ruta más corta entre un nodo origen y todos los demás nodos en un grafo ponderado con pesos no negativos. Su complejidad temporal es $O((V + E) \log V)$ con una cola de prioridad.

En este proyecto, el campus se modela como un grafo no dirigido $G = (V, E)$ donde:

- $V = \{v_1, v_2, \dots, v_9\}$ representan 9 ubicaciones del campus
- $E = \{e_1, e_2, \dots, e_{13}\}$ representan 13 conexiones peatonales
- $w(e_i)$ es la distancia en metros de cada arista

3. Metodología

3.1 Recopilación de Datos

Se capturaron más de **1,236 imágenes** del campus de la UAO distribuidas en 6 clases:

Clase	Descripción	Imágenes aprox.
Arco	Entrada principal del campus	~206
Aulas 1	Primer bloque de aulas	~206
Aulas 2	Segundo bloque de aulas	~206
Sótano 1	Primer nivel subterráneo	~206
Sótano 2	Segundo nivel subterráneo	~206
Cafetería	Zona de alimentación	~206

Las imágenes fueron anotadas en formato PASCAL VOC con bounding boxes para cada punto de referencia.

3.2 Entrenamiento del Modelo

1. **Preprocesamiento:** Redimensionamiento a 640×640, augmentación (rotación, flip horizontal, variación de brillo)
2. **Transfer Learning:** Inicialización con pesos pre-entrenados en COCO
3. **Fine-tuning:** Entrenamiento por 50 épocas con learning rate 0.001, batch size 8

4. **Evaluación:** mAP@0.5, precision, recall por clase

3.3 Modelado del Grafo del Campus

El grafo del campus contiene:

- **6 nodos principales** (puntos de referencia detectables)
- **3 nodos intermedios** (Plazoleta Central, Biblioteca, Lago)
- **13 aristas** con pesos en metros

3.4 Implementación de la Aplicación Móvil

Desarrollada con React Native + Expo + TypeScript:

- Módulo de visión con interfaz intercambiable (mock/real)
- Algoritmo de Dijkstra en TypeScript puro
- Mapa SVG interactivo con react-native-svg
- Navegación por pestañas con expo-router

4. Arquitectura del Sistema

El sistema sigue una arquitectura modular:

1. **Capa de Presentación:** Pantallas y componentes React Native
2. **Capa de Servicios:** Módulos de visión (IVisionDetector) y pathfinding (Dijkstra)
3. **Capa de Datos:** Definiciones de landmarks y grafo del campus
4. **Capa de Estado:** NavigationContext para estado compartido

5. Resultados

Sección pendiente de resultados experimentales con el modelo real.

Métricas esperadas:

- mAP@0.5 del modelo Faster R-CNN
- Tiempo de inferencia en dispositivo (ms)
- Precisión de navegación vs. rutas reales
- Satisfacción de usuario (encuesta SUS)

6. Conclusiones

UAO Finder demuestra la viabilidad de combinar técnicas de visión por computadora con algoritmos clásicos de grafos para crear sistemas de navegación indoor en entornos universitarios. La arquitectura modular permite la evolución gradual del sistema, desde la simulación actual hasta la integración de modelos reales de TFLite.

7. Referencias

1. Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), 1137-1149.
2. Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1), 269-271.
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR* 2016.
4. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR* 2018.
5. **UVManos** — Sistema de reconocimiento de gestos de manos, Universidad del Valle, Cali, Colombia. Referencia metodológica para pipelines de visión por computadora aplicados a reconocimiento en entornos universitarios. Demuestra la viabilidad de modelos de deep learning para clasificación visual en campus colombianos.
6. **Mcity** — Instalación de prueba de vehículos autónomos, University of Michigan. Referencia para el enfoque de navegación a escala campus que combina percepción visual con algoritmos de búsqueda de caminos en entornos controlados y mapeados. Inspiró el diseño del grafo peatonal y la integración percepción-navegación.
7. Lin, T.-Y., et al. (2014). Microsoft COCO: Common Objects in Context. *ECCV* 2014.
8. Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv:1704.04861*.